

# MATH 231 – Linear Algebra I

## Homework 1 • Vectors and Complex Numbers

Kiely Hall 326 • Instructor: Kevin Bedoya

**Due:** to be announced on the class Discord.

| Lecture             | Sections | Topics                                                                                                                                                                                                                                                                   |
|---------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vector Unit, Part 1 | §1–§9    | magnitude and direction; the Euclidean norm; vector vs. scalar quantities; notation, $\mathbb{R}^2$ , and signatures; addition and subtraction; the parallelogram rule and the triangle inequality; scalar multiplication; the eight algebraic laws; equality of vectors |
| Complex Numbers     | §1–§7    | the imaginary unit; $a + bi$ , real and imaginary parts; the complex plane; addition, multiplication, the conjugate, division; the modulus, distance, and discs; the unit circle, trigonometric form, Euler’s formula; multiplication by $i$ as a rotation               |

### What to submit

Push to your own MATH 231 GitHub repository, in a folder named `homework/hw1/`:

- **Parts A–D** — your written work, as a single PDF named `hw1.pdf`. Handwritten and scanned or photographed is completely fine, as long as it is legible and the pages are in order.

**PDF is the only accepted format for the written part.** A `.docx`, a `.jpg`, a `.png`, a `.heic`, or a link to a file hosted somewhere else will not be graded. Phone scanning apps export PDF directly, and every word processor can “Save as PDF”.

**Type Parts A–D up instead and you earn +5 points of extra credit on this assignment.** Any typesetting system counts — L<sup>A</sup>T<sub>E</sub>X, Word’s equation editor, or anything else that produces readable mathematics — and the extra credit is for the whole of Parts A–D, not for typing up some of it. The file you submit must still be a PDF.

- **Part E** — two Python files, `hw1.py` (your work, one function per question part) and `runhw1.py` (which imports those functions and runs them all, labelling each answer). Do not submit a copy of your output: `runhw1.py` is **run as submitted**, and Part E is graded from what it prints. *How to structure your submission*, in Part E, specifies both files in full.

**Python is the only language accepted for the programming.** Any other language earns no credit for Part E, however correct it is.

If you have not set up the repository yet, Setup Guide 1 (§ “Create your folders” and § “Save your work and send it to GitHub”) is the walkthrough. Only pushed work can be graded.

### Label every answer

**Parts A–D.** Label each answer with the number of the problem it answers — A.2(f), C.5(d), D.1(b) — and put your answers **in the same order as they appear in this assignment**. A grader who cannot tell which question a piece of work belongs to cannot award it credit, and that is the most common way a correct answer loses points.

**You do not have to copy the question out.** This applies to handwritten and typed submissions alike: the label alone is enough. Write A.2(f), then your work. Do not reproduce the wording of the problem.

**Part E.** The labelling is built into the file structure specified under *How to structure your submission*: one function per question part in `hw1.py`, named after that part, with a comment naming it directly above; and one labelled line per part in `runhw1.py`. Code that runs correctly but cannot be matched to a question is graded the same way an unlabelled answer sheet is.

**Grading.** Every problem is worth the same number of points. Part E is graded by running `runhw1.py` and reading what it prints — on whether it produces correct answers, not on programming style.

**Use Part E to check Parts A and C.** Every numerical answer in those two parts can be verified in two lines of NumPy, and you are encouraged to check all of them before submitting — that is what the tool is for. Two limits. The hand work must still be shown: a correct number with no derivation earns partial credit at best. And no proof in Part B or Part D can be checked this way, since code confirms a statement only for the numbers you tried, not for all of them.

**On the proof problems (Parts B and D).** These are short, and none of them requires a clever idea. What they require is that you write down what you are assuming, work in components or from the stated laws, justify each equality by naming the fact that licenses it, and end with a sentence saying what you have shown. A chain of equalities with no words is not a proof; neither is a paragraph of words with no computation. Two or three sentences and three or four lines of algebra is the expected length.

**Collaboration** is encouraged — work at a board together, argue about the problems. What you submit must be written by you, in your own words and your own code.

### Part A. Vectors — computation

*Vector Unit, Part 1.* Graph paper helps for the drawing parts; neat freehand axes are acceptable. Leave norms in exact form ( $\sqrt{13}$ ,  $3\sqrt{5}$ ) unless a decimal is asked for — a norm is generically irrational, and  $\sqrt{13}$  is a finished answer.

**Problem A.1** (Reading a vector off an arrow). Each row below describes an arrow in the plane by its tail and its head.

|     | Tail    | Head    |
|-----|---------|---------|
| (a) | (0, 0)  | (5, 12) |
| (b) | (2, -1) | (6, 2)  |
| (c) | (-3, 4) | (1, 7)  |
| (d) | (1, 1)  | (-2, 5) |

- (i) Write each arrow as a vector in bracket notation, using *head minus tail*.
- (ii) Compute the norm of each.
- (iii) Exactly one pair among (a)–(d) consists of two arrows that are the *same vector*. Which pair, and why? Draw both arrows on one set of axes to show it.
- (iv) Three of the four have the same norm. Explain in one sentence why that does *not* make them the same vector, citing the definition of equality of vectors.

**Problem A.2** (Componentwise arithmetic). Let

$$\mathbf{u} = [3, -4], \quad \mathbf{v} = [-1, 2], \quad \mathbf{w} = [5, 0], \quad \alpha = -3, \quad \beta = \frac{1}{2}.$$

Compute each of the following. Show the componentwise step, not just the answer.

- (a)  $\mathbf{u} + \mathbf{v}$
- (b)  $\mathbf{u} - \mathbf{v}$
- (c)  $\mathbf{v} - \mathbf{u}$
- (d)  $\alpha \mathbf{u}$
- (e)  $\beta \mathbf{v} + \mathbf{u}$
- (f)  $2\mathbf{u} - 3\mathbf{v} + \mathbf{w}$
- (g)  $\|\mathbf{u}\|, \|\mathbf{v}\|, \|\mathbf{w}\|$
- (h)  $\|\mathbf{u} + \mathbf{v}\|$
- (i)  $\|\alpha \mathbf{u}\|$ , and the value of  $|\alpha| \|\mathbf{u}\|$  — do they agree?

**Problem A.3** (The triangle inequality by hand). For each pair, compute  $\|\mathbf{u} + \mathbf{v}\|$  and  $\|\mathbf{u}\| + \|\mathbf{v}\|$ , and say whether the triangle inequality holds *strictly* ( $<$ ) or with *equality* ( $=$ ).

|     | $\mathbf{u}$ | $\mathbf{v}$ |
|-----|--------------|--------------|
| (a) | $[3, 0]$     | $[0, 4]$     |
| (b) | $[2, 3]$     | $[4, 6]$     |
| (c) | $[1, -2]$    | $[-2, 4]$    |
| (d) | $[2, 1]$     | $[0, 0]$     |

**Problem A.4** (Reading the scalar). Let  $\mathbf{v} = [2, -3]$ .

- (a) Write the vector on the same line through the origin as  $\mathbf{v}$  that points the *same* way and is *four times* as long.

- (b) Write the vector that points the *opposite* way and is *half* as long.
- (c) Write the mirror image of  $\mathbf{v}$  through the origin. What is its norm, and why could you have said so without computing?
- (d) Is  $[3, -4.5]$  a scalar multiple of  $\mathbf{v}$ ? Is  $[4, -6.1]$ ? Justify each answer.
- (e) What does  $0\mathbf{v}$  equal, and what happens to the direction?

**Problem A.5** (Using the eight laws). Let  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$ . Simplify each expression to the form  $\alpha\mathbf{u} + \beta\mathbf{v}$  for real numbers  $\alpha, \beta$ . At every step, name the law (**A1–A4**, **S1–S4**) that licenses it.

- (a)  $3(\mathbf{u} + 2\mathbf{v}) - 2(\mathbf{u} - \mathbf{v})$
- (b)  $-(\mathbf{u} - \mathbf{v}) + 4(\mathbf{v} + \frac{1}{2}\mathbf{u})$
- (c)  $5(\mathbf{u} + \mathbf{v}) - 5\mathbf{u} - 5\mathbf{v}$

Then substitute  $\mathbf{u} = [1, -2]$  and  $\mathbf{v} = [3, 1]$  into both the original expression and your simplified form for each of (a), (b), (c), and confirm they agree.

**Watch out.** None of the eight laws is a statement about *subtraction*, so you cannot cite one to justify a step taken on a subtraction sign directly. Rewrite every subtraction as  $\mathbf{u} + (-\mathbf{v})$  first.

**Problem A.6** (Signatures). Give the signature  $f : A \rightarrow B$  of each operation below, using the placeholder-dot convention, and say whether it is *unary* or *binary*.

- (a) the Euclidean norm on  $\mathbb{R}^2$ ;
- (b) addition of two vectors in  $\mathbb{R}^2$ ;
- (c) scalar multiplication;
- (d) the operation sending a vector to its mirror image through the origin;
- (e) the operation sending a pair of vectors  $\mathbf{u}, \mathbf{v}$  to the distance between their heads, that is, to  $\|\mathbf{u} - \mathbf{v}\|$ .

Then: two of the five above have the *same* signature but are different operations. Which two, and what does that tell you about how much information a signature carries?

**Problem A.7** (Vector or scalar?). Classify each quantity as a **vector quantity** or a **scalar quantity**, and for each vector quantity say in a few words what its direction means physically.

mass · velocity · speed · force · temperature · temperature gradient · work · momentum · electric charge · displacement · distance travelled · acceleration

- (a) A car drives once around a circular track at a constant 40 mph. Is its speed constant? Is its velocity? Is it accelerating?
- (b) The same car finishes where it started. What was its total *displacement*, and what was its total *distance travelled*? Explain why the two answers differ.

## Part B. Vectors — proof

*Vector Unit, Part 1.* Every one of these is proved the same way the lecture proves **A1**, **S1** and **A4**: write the vectors in components, use the corresponding fact about *real* numbers one coordinate at a time, and reassemble — except for Problem B.4, which deliberately forbids components.

**Problem B.1** (Reversing a difference). Let  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$ .

(a) Prove that  $\mathbf{u} - \mathbf{v} = -(\mathbf{v} - \mathbf{u})$ .

(b) Prove that subtraction is not associative by showing that

$$(\mathbf{u} - \mathbf{v}) - \mathbf{w} = [\mathbf{u} - (\mathbf{v} - \mathbf{w})] - 2\mathbf{w}$$

for all  $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^2$ , and give explicit  $\mathbf{u}, \mathbf{v}, \mathbf{w}$  for which the two bracketings genuinely differ. Then state the one condition on  $\mathbf{w}$  under which they agree, and prove it.

**Problem B.2** (Two of the eight laws). Let  $\mathbf{u} \in \mathbb{R}^2$  and  $\alpha, \beta \in \mathbb{R}$ . Prove, in components:

(a) **S2**:  $(\alpha + \beta)\mathbf{u} = \alpha\mathbf{u} + \beta\mathbf{u}$ .

(b) **S3**:  $\alpha(\beta\mathbf{u}) = (\alpha\beta)\mathbf{u}$ .

For each, name the property of the *real numbers* that your proof ultimately rests on. Then explain, in one sentence, why **S1** and **S2** are genuinely different statements rather than the same distributive law written twice.

**Problem B.3** (Scaling multiplies the length by  $|\alpha|$ ). Prove that for every  $\alpha \in \mathbb{R}$  and every  $\mathbf{v} \in \mathbb{R}^2$ ,

$$\|\alpha\mathbf{v}\| = |\alpha| \|\mathbf{v}\|.$$

**Watch out.** At one point you will meet  $\sqrt{\alpha^2}$ . It equals  $|\alpha|$ , *not*  $\alpha$ . Say why in your write-up, and check your claim against the case  $\alpha = -3$ ,  $\mathbf{v} = [1, 0]$  — if you had written  $\alpha$  there, what would the norm of  $[-3, 0]$  have come out to be, and why is that impossible?

**Problem B.4** (Working from the laws alone, without components). This problem is about what the eight laws can do *by themselves*. For this problem only, you may not write a vector in components: you may use only **A1–A4**, **S1–S4**, the uniqueness of the additive inverse, and ordinary arithmetic on the real scalars.

Let  $\mathbf{v} \in \mathbb{R}^2$ .

(a) Prove that  $0\mathbf{v} = \mathbf{0}$ . That is: the *scalar* zero times any vector is the *zero vector*.

**Hint.** Start from  $0\mathbf{v} = (0 + 0)\mathbf{v}$  and apply **S2**. You now have  $\mathbf{x} = \mathbf{x} + \mathbf{x}$  for  $\mathbf{x} = 0\mathbf{v}$ ; finish by adding  $-\mathbf{x}$  to both sides and using **A4**, **A2**, **A3**.

(b) Using (a), **S2** and **S4**, prove that  $\mathbf{v} + (-1)\mathbf{v} = \mathbf{0}$ .

- (c) Conclude that  $(-1)\mathbf{v} = -\mathbf{v}$ , where  $-\mathbf{v}$  denotes the additive inverse guaranteed by **A4**. Say explicitly where you are using the fact that the additive inverse is *unique* — without it, (b) would only show that  $(-1)\mathbf{v}$  is *an* inverse, not *the* inverse.

Finally, in one or two sentences: why is it worth proving  $(-1)\mathbf{v} = -\mathbf{v}$  this way, when a single line in components would also do it?

**Problem B.5** (When the triangle inequality is an equality). Prove: if  $\mathbf{v} = \alpha\mathbf{u}$  for some scalar  $\alpha \geq 0$ , then

$$\|\mathbf{u} + \mathbf{v}\| = \|\mathbf{u}\| + \|\mathbf{v}\|.$$

**Hint.** Write  $\mathbf{u} + \mathbf{v} = \mathbf{u} + \alpha\mathbf{u}$  and use **S2** and **S4** to collect it into a single scalar multiple of  $\mathbf{u}$ . Then apply Problem B.3. You will need  $|1 + \alpha| = 1 + \alpha$  — say why the hypothesis  $\alpha \geq 0$  is what makes that true.

Then: where exactly does your proof break down if  $\alpha < 0$ ? Give a concrete  $\mathbf{u}$  and  $\alpha < 0$  for which the two sides are unequal, and identify which step fails.

## Part C. Complex numbers — computation

*Complex Numbers.* Leave answers in the form  $a + bi$  with  $a$  and  $b$  real, in lowest terms. A quotient is not finished until it is written that way — that is the whole point of the division rule.

**Problem C.1** (Parts, and the plane). For each of

$$z_1 = 4 - 3i, \quad z_2 = -2 + 5i, \quad z_3 = 6, \quad z_4 = -i, \quad z_5 = \frac{1}{2} + \frac{3}{2}i,$$

write down  $\operatorname{Re}(z)$  and  $\operatorname{Im}(z)$ , and plot all five on one copy of the complex plane, labelled.

Then: which of the five lie on the real axis? Which on the imaginary axis? And which of them are ordinary real numbers?

**Watch out.**  $\operatorname{Im}(4 - 3i)$  is the real number  $-3$ , not  $-3i$  and not  $3$ . Go back over the definition of the imaginary part if that surprised you.

**Problem C.2** (The four operations). Let  $z = 3 - 2i$  and  $w = -1 + 4i$ . Compute, showing your work:

(a)  $z + w$       and       $z - w$       and       $w - z$

(b)  $zw$ , twice: once by expanding the binomials and using  $i^2 = -1$ , and once by substituting into the formula  $(a + bi)(c + di) = (ac - bd) + (ad + bc)i$ . The two must agree.

(c)  $z^2$

(d)  $\bar{z}$ , and  $z\bar{z}$

(e)  $\frac{z}{w}$ , by multiplying through by  $\bar{w}/\bar{w}$

(f)  $\frac{1}{w}$

- (g)  $|z|$ ,  $|w|$ , and  $|zw|$ . Compare  $|zw|$  with  $|z| \cdot |w|$  and state what you notice. (You will prove it in Problem D.3.)

**Problem C.3** (Powers of  $i$ ). Compute, in the form  $a + bi$  with  $a$  and  $b$  real:

$$i^3, \quad i^4, \quad i^5, \quad i^{10}, \quad i^{27}, \quad i^{100}, \quad (-i)^3, \quad (2i)^4.$$

The first seven each come out to one of  $1, -1, i, -i$ . The last one does not — say why not, and what the extra factor is.

Then: explain in one or two sentences why the powers of  $i$  repeat with period 4, and describe what that repetition looks like on the complex plane. Finally, give a rule that computes  $i^n$  for any positive integer  $n$  from the remainder of  $n$  on division by 4.

**Problem C.4** (The conjugate). Let  $z = 5 - 12i$ .

- (a) Compute  $\bar{z}$ ,  $z\bar{z}$ , and  $|z|$ . Verify that  $|z|^2 = z\bar{z}$ .
- (b) Compute  $z + \bar{z}$  and  $z - \bar{z}$ . Express each in terms of  $\operatorname{Re}(z)$  and  $\operatorname{Im}(z)$ , and guess the general identity. (You will prove it in Problem D.1.)
- (c) Plot  $z$  and  $\bar{z}$  and describe the geometric relationship between them.
- (d) Compute  $\bar{\bar{z}}$ . What does conjugating twice do, and why is that obvious from the picture?
- (e) Find every complex number  $z$  satisfying  $z = \bar{z}$ . Describe the answer as a subset of the complex plane.

**Problem C.5** (Modulus, distance, and discs). (a) Compute  $|-3 + 4i|$ ,  $|1 + i|$ ,  $|-7|$ ,  $|2i|$ , and  $|0|$ .

- (b) Compute the distance from  $3 + 2i$  to  $-1 - i$ , and the distance from  $5 - i$  to  $5 + 3i$ . For the second, explain why you could have read the answer off the picture without computing.
- (c) Describe in words, and sketch, the set of complex numbers satisfying each condition:

$$|z| < 2, \quad |z - (2 - i)| = 3, \quad |z - 4i| \leq 1.$$

Say for each whether the boundary circle is included.

- (d) Write down an inequality describing the closed disc of radius 5 centred at  $-3 + 2i$ .
- (e) Does  $1 + i$  lie in the disc  $|z - (2 - i)| \leq 3$ ? Decide by computing, and say how far inside or outside it is.
- (f) Describe the set  $|z| = 0$ . How many complex numbers are in it?

**Problem C.6** (The unit circle, trigonometric form, and Euler). (a) By computing the modulus of each, verify that all four of the complex numbers

$$i, \quad \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i, \quad \frac{1}{2} + \frac{\sqrt{3}}{2}i, \quad -\frac{3}{5} + \frac{4}{5}i$$

lie on the unit circle.

(b) Write each of the four in trigonometric form  $\cos \theta + i \sin \theta$ , with  $\theta \in [0, 2\pi)$ . The first three have exact angles; for the fourth, give  $\theta$  to four decimal places (a calculator is fine, or use `np.angle` from Part E) and say which quadrant it is in.

(c) Use Euler's formula to evaluate, in the form  $a + bi$ :

$$e^{i\pi/2}, \quad e^{i\pi}, \quad e^{2\pi i}, \quad e^{i\pi/4}, \quad e^{-i\pi/2}.$$

(d) State Euler's identity, and say which of your answers in (c) it is a rearrangement of.

(e) A point of the unit circle needs only *one* real number to pin it down, where a general complex number needs two. Explain that in one or two sentences, and say what the missing second number is being fixed to.

**Problem C.7** (Multiplication by  $i$  as a rotation). Let  $z = 3 + 2i$ .

(a) Compute  $iz$ ,  $i^2z$ ,  $i^3z$  and  $i^4z$ , and plot all five points  $z, iz, i^2z, i^3z, i^4z$  on one complex plane. Join them in order. What shape do the four distinct points form?

(b) Compute  $|iz|$  and compare it with  $|z|$ . Why is this worth checking, rather than stopping once the angle is known to be  $90^\circ$ ?

(c) Compute  $(-i)z$ . Which way did the point turn this time?

(d) A point sits at  $(4, 1)$  in the plane. Encode it as a complex number and rotate it  $90^\circ$  counter-clockwise about the origin by a single multiplication. Give the coordinates of the rotated point.

(e) Rotate the same point by  $180^\circ$ , again by a single multiplication. Which complex number do you multiply by, and why?

(f) Take an arbitrary  $z = a + bi$  and compute  $iz$  symbolically. Write both  $z$  and  $iz$  as vectors in  $\mathbb{R}^2$ . The full argument finishes with a dot product — an operation we meet in Part 2 of the vector unit. *Without* using it, describe in words the relationship between  $[a, b]$  and  $[-b, a]$ : what happened to the two coordinates, and what does that do to the arrow?

## Part D. Complex numbers — proof

*Complex Numbers.* In every problem, write  $z = a + bi$  and  $w = c + di$  with  $a, b, c, d \in \mathbb{R}$  and compute. As in Part B, name the fact about real numbers that each step relies on when it is not obvious.

**Problem D.1** (Recovering the parts from the conjugate). Let  $z \in \mathbb{C}$ . Prove:

(a)  $z + \bar{z} = 2 \operatorname{Re}(z)$ ;

(b)  $z - \bar{z} = 2i \operatorname{Im}(z)$ ;

(c)  $z$  is real if and only if  $z = \bar{z}$ ;

(d)  $\operatorname{Re}(z) = 0$  if and only if  $z = -\bar{z}$ .

Parts (c) and (d) are “if and only if” statements, so each needs *both* directions argued. Say explicitly which direction you are proving as you prove it.

**Problem D.2** (Conjugation respects the arithmetic). Let  $z, w \in \mathbb{C}$ . Prove:

(a)  $\overline{z + w} = \bar{z} + \bar{w}$ ;

(b)  $\overline{zw} = \bar{z}\bar{w}$ .

For (b), expand both sides using the product formula and compare real and imaginary parts. Then state, in one sentence, what (b) says geometrically: if reflecting in the real axis is what conjugation *does*, what have you just shown about reflecting a product versus multiplying two reflections?

**Problem D.3** (The modulus is multiplicative). Prove that  $|zw| = |z||w|$  for all  $z, w \in \mathbb{C}$ .

(a) Show  $|zw|^2 = (ac - bd)^2 + (ad + bc)^2$  and expand it fully.

(b) Show that the expansion factors as  $(a^2 + b^2)(c^2 + d^2)$ . Note which two terms cancel — that cancellation is the entire content of the identity.

**Problem D.4** (The unit circle really is the unit circle). Let  $\theta$  be any real number. Prove:

(a)  $|\cos \theta + i \sin \theta| = 1$ . Name the trigonometric identity you use.

(b)  $|e^{i\theta}| = 1$ , deduced from (a) and Euler’s formula.

**Problem D.5** (Multiplying by a point of the unit circle preserves distance). Let  $z \in \mathbb{C}$  and let  $\theta \in \mathbb{R}$ .

(a) Prove that  $|z e^{i\theta}| = |z|$ .

**Hint.** Problem D.3 plus Problem D.4(b). This is a two-line proof.

(b) Deduce as a special case that  $|iz| = |z|$  for all  $z$ , by identifying the value of  $\theta$  for which  $e^{i\theta} = i$ .

## Part E. NumPy

*Required.*

This part is a tutorial, then a specification of the two files you hand in, then the problems. Read the tutorial sections typing the code in as you go, read *How to structure your submission* before you write anything you intend to submit, then do Problems E.1–E.8. If you have never programmed before, that is expected and fine: everything you need is below, and nothing here is harder than the arithmetic you did by hand in Parts A and C.

Two references live alongside this assignment in the course repository: **Setup Guide 2** (installing Python, pip, and an IDE) and the **Python Recipe Book** ([lectures/python-recipe/](#)), which is a lookup table of the calls covered here and in later lectures. Use the recipe book as a dictionary, not a textbook.

## E.1. Getting set up

Install once:

```
python -m pip install numpy
```

Every block below assumes this line at the top of your file:

```
import numpy as np
```

**Checkpoint.** Create `hw1.py`, put the import line in it, add `print(np.array([1.0, 2.0]))`, and run it. You should see `[1. 2.]`. If you get `ModuleNotFoundError: No module named 'numpy'`, the install went to a different Python than the one running your file — [Setup Guide 2 §8](#) covers this exact failure.

## E.2. Vectors: making them, and reading them

A vector in  $\mathbb{R}^2$  is a NumPy array of two numbers.

```
import numpy as np

u = np.array([3.0, -4.0])      # the vector [3, -4]
v = np.array([-1.0, 2.0])     # the vector [-1, 2]

print(u[0])                   # 3.0    -- the first component
print(u[1])                   # -4.0   -- the second
print(len(u))                 # 2      -- the n in  $\mathbb{R}^n$ 
print(u.shape)                # (2,)
```

**Watch out. Indices start at zero.** The first component of `u` is `u[0]`, not `u[1]`. Lecture counts from 1; Python counts from 0. Subtract one.

**Watch out. Write 3.0, not 3, when the answers will not be whole numbers.** An array built from integers stays an integer array, and some operations will surprise you. Using floats throughout this assignment costs nothing and avoids the issue entirely.

## E.3. Addition, subtraction, and scaling

NumPy spells these exactly as the lecture does.

```
u + v          # array([ 2., -2.]) -- componentwise addition
u - v          # array([ 4., -6.]) -- componentwise subtraction
-3 * u         # array([-9., 12.]) -- scalar multiplication
-u            # array([-3.,  4.])  -- the mirror image,  $(-1)u$ 
2*u - 3*v     # array([ 9., -14.]) -- combinations, as in Problem A.5
```

**Watch out. A Python list is not a vector.** Ordinary square brackets with no `np.array` around them make a *list*, and `+` on two lists means *glue them end to end*:

```
[1, 2] + [3, 4]          # [1, 2, 3, 4] -- concatenation!
np.array([1, 2]) + np.array([3, 4]) # array([4, 6]) -- what you wanted
```

Every vector in this assignment must be wrapped in `np.array`.

**Watch out.**  $u * v$  is not a vector operation we have defined. NumPy will happily compute it — it multiplies entry by entry — but that is not addition, not scaling, and not any operation from the lectures. Do not use it in this assignment.

#### E.4. The norm

```
np.linalg.norm(u)           # 5.0 -- the Euclidean norm ||u||
np.sqrt(u[0]**2 + u[1]**2)   # 5.0 -- the same thing, by Pythagoras
np.linalg.norm(u - v)        # the distance between the two heads
```

The name is worth parsing: `np` is NumPy, `linalg` is its linear-algebra module, `norm` is the function. Read it as “NumPy’s linear algebra norm”.

#### E.5. Comparing floating-point numbers

This is the one piece of programming discipline this assignment insists on.

```
0.1 + 0.2 == 0.3             # False    (!)
0.1 + 0.2                     # 0.30000000000000004
np.isclose(0.1 + 0.2, 0.3)    # True     -- the right way to compare
```

Computers store real numbers to about sixteen significant digits, so two quantities that are mathematically equal can differ in the last few. Testing them with `==` then reports `False` and tells you nothing.

#### The rule for this course

Never compare computed floating-point values with `==`. Use `np.isclose(x, y)` for two numbers, and `np.allclose(a, b)` for two arrays. Both work on complex numbers too.

#### E.6. Complex numbers in Python

```
z = 3 + 4j                    # NOT 3 + 4i -- Python writes the imaginary unit as j
w = complex(1, -2)           # the same as 1 - 2j

z.real                        # 3.0          -- Re(z), a real number
z.imag                        # 4.0          -- Im(z), a real number
z.conjugate()                 # (3-4j)       -- the conjugate, z-bar
abs(z)                        # 5.0          -- the modulus |z|

z + w                         # (4+2j)
z * w                         # (11-2j)
z / w                         # (-1+2j)
z ** 2                        # (-7+24j)

np.angle(z)                   # 0.9272952...  -- the angle theta, in radians
np.exp(1j * np.pi / 2)       # ~ (6.1e-17+1j) -- Euler's formula: e^{i pi/2} = i
```

**Watch out.** The imaginary unit is `j`, and it must carry a coefficient. Writing `j` alone is a variable name and will raise `NameError`; the imaginary unit is `1j`. (Python borrowed `j` from electrical engineering, where  $i$  already means current.) Also note `1j**2` prints as `(-1+0j)`, not `-1` — the value is  $-1$ , but the type stays complex.

**Watch out.** `z.imag` is a real number. For `z = 3 + 4j` it is `4.0`, not `4j` — which is exactly the convention used in lecture. `.real` and `.imag` are attributes, so they take no parentheses; `.conjugate()` is a method, so it does.

**Watch out.** The last line of the block above prints `(6.123233995736766e-17+1j)`, not a clean `1j`. That  $6.1 \times 10^{-17}$  is floating-point error, not mathematics:  $\cos(\pi/2)$  is zero but the stored value of  $\pi$  is not exactly  $\pi$ . This is why `np.isclose` exists, and Problem E.6 asks you to think about it.

## E.7. How to structure your submission

Part E is submitted as **two** Python files with different jobs.

- **hw1.py — your work.** One function per lettered part, named `q number letter`: Problem E.2(a) becomes `q2a`, Problem E.6(c) becomes `q6c`. A question with no lettered parts gets one function named for the question alone, so E.8 becomes `q8`. Each function **returns** its answer; when an answer has several pieces, return them together as a tuple or a list.
- **runhw1.py — the driver.** It imports those functions from `hw1.py` and calls them in order, one line per part, printing a label before each answer. It contains no mathematics of its own. **This is the file that is run to grade Part E.**

**hw1.py — definitions only.** Nothing may run when the file is imported: define your functions and stop. A `print` sitting at the left margin fires the moment `runhw1.py` imports the file, and its output lands above the labels, where it cannot be matched to a question.

```
# hw1.py -- the work.  Definitions only; nothing runs on import.

import numpy as np

u = np.array([3.0, -4.0])
v = np.array([-1.0, 2.0])

def norm2(vec):                                # helpers are fine, and may be shared
    return np.sqrt(vec[0]**2 + vec[1]**2)

# --- E.2(a) -----
def q2a():
    return norm2(u)

# --- E.2(b) -----
def q2b():
    return [bool(np.isclose(norm2(w), np.linalg.norm(w))) for w in (u, v)]
```

**runhw1.py — one line per part.** Import the functions from their source file, then call them in the order the questions appear. Each line prints a label followed by the answer that call produced:

```
# runhw1.py -- runs every question in order.  No mathematics in this file.

import hw1
```

```
print("Output of question 2 part a : ", hw1.q2a())
print("Output of question 2 part b : ", hw1.q2b())
print("Output of question 2 part c : ", hw1.q2c())
```

(from hw1 import q2a, q2b, q2c and then q2a() is equally acceptable if you prefer it.)

The label is required, and its wording is fixed:

```
"Output of question <number> part <letter> : "
```

So Problem E.6(c) is labelled `Output of question 6 part c : .` For a question with no lettered parts, drop that half — `"Output of question 8 : "`.

**Watch out.** Several problems below say “print” something. Return it instead — `runhw1.py` does all the printing, and that is where the value shows up under its label. Nothing in `hw1.py` should print.

**Checkpoint.** Run `runhw1.py` once before you push and read the output top to bottom. Every part of every question should appear, in order, under its own label, with no error and nothing unlabelled.

## E.8. The problems

**Problem E.1** (Checking Part A). Enter  $\mathbf{u} = [3, -4]$ ,  $\mathbf{v} = [-1, 2]$ ,  $\mathbf{w} = [5, 0]$  as NumPy arrays and print the value of every one of the nine sub-parts (a)–(i) of Problem A.2. Compare each against what you computed by hand and state, in a comment, whether they agree. If any disagree, find the error — it is far more often in the hand computation than in the code, and finding it is the point of the exercise.

**Problem E.2** (Writing the norm yourself). `np.linalg.norm` is a black box. Open it.

- Write a function `norm2(v)` that takes a two-component array and returns  $\sqrt{x^2 + y^2}$ , using only `np.sqrt`, indexing, and arithmetic — not `np.linalg.norm`.
- Test it against `np.linalg.norm` on all four vectors of Problem A.1, using `np.isclose`, and return the four verdicts.
- Use `norm2` to give the norms of the four vectors from Problem A.1 as decimals, and check them against the exact values you wrote by hand.

```
def norm2(v):
    # your two lines here
    ...

# --- E.2(b) -----
def q2b():
    return [bool(np.isclose(norm2(w), np.linalg.norm(w))) for w in vectors]
```

**Problem E.3** (Head minus tail). Write a function `vector_from(tail, head)` that takes two points, given as NumPy arrays, and returns the vector they represent — head minus tail.

Use it on all four rows of the table in Problem A.1, return the four results, and then use `np.allclose` to confirm in code which pair are the same vector. Return that answer as a sentence, for example "rows (x) and (y) are the same vector: True".

**Problem E.4** (Testing the triangle inequality at scale). By hand you checked four pairs. Check ten thousand.

- (a) Using the generator below, draw 10,000 random pairs  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^2$  and count how many satisfy  $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ . Print the count.
- (b) In the same loop, count how many satisfy it with *equality*, in the `np.isclose` sense. Print that count too, and explain the number you get in one sentence.
- (c) Now construct, by hand in code, one pair for which equality *does* hold, and verify it with `np.isclose`. Say what it is about your pair that forces equality.
- (d) Answer in a comment: your run in (a) found no counterexample. Does that *prove* the triangle inequality? Say why or why not, in one or two sentences.

```
rng = np.random.default_rng(231)      # a fixed seed, so your run is reproducible

for _ in range(10000):
    u = rng.normal(size=2)             # a random vector in R^2
    v = rng.normal(size=2)
    # your test here
```

**Problem E.5** (Implementing complex arithmetic from the formulas). Python already multiplies and divides complex numbers. Ignore that, and implement the multiplication and division rules yourself.

Write three functions that take the four *real* numbers  $a, b, c, d$  and return the answer as a Python complex number, using only real arithmetic internally and the formulas from the lecture:

- (a) `cmul(a, b, c, d)`, computing  $(a + bi)(c + di)$  by the boxed formula  $(ac - bd) + (ad + bc)i$ ;
- (b) `cdiv(a, b, c, d)`, computing  $(a + bi)/(c + di)$  by the conjugate trick, and raising an error if  $c$  and  $d$  are both zero;
- (c) `cmod(a, b)`, computing  $|a + bi|$  from Pythagoras.

Then test all three against Python's built-in `*`, `/` and `abs` on the pair  $z = 3 - 2i$ ,  $w = -1 + 4i$  from Problem C.2, using `np.isclose`, and return the verdicts. Finally, call `cdiv` with  $c = d = 0$  inside a `try` block, catch the error, and return its message, to show the guard works.

```
def cmul(a, b, c, d):
    return complex(..., ...)          # (ac - bd) + (ad + bc)i
```

**Watch out.** Build the answer with `complex(real_part, imag_part)`. If you find yourself writing `(a + b*1j) * (c + d*1j)` you have used Python's multiplication instead of implementing the lecture's formula, which is the one thing this problem is asking you not to do.

**Problem E.6** (Euler’s formula, numerically). (a) For each of the ten values  $\theta = 0, \pi/10, 2\pi/10, \dots, 9\pi/10$ , compute `np.exp(1j*theta)` and `np.cos(theta) + 1j*np.sin(theta)`, and confirm with `np.isclose` that they agree. Return the ten  $\theta$  values paired with their ten verdicts. The block at the end of this problem builds all ten values of  $\theta$  for you in one line.

(b) Print `np.exp(1j*np.pi) + 1`. It is not exactly zero. Print `np.isclose(np.exp(1j*np.pi) + 1, 0)` as well. In a comment, explain in two sentences why the exact answer is 0 (Euler’s identity) but the computed answer is not, and what the size of the discrepancy tells you.

(c) Confirm that  $|e^{i\theta}| = 1$  for all ten values of  $\theta$  from (a), using `np.abs` and `np.allclose`. This is Problem D.4(b), checked numerically — note again that checking it on ten values is not what proving it means.

(d) Verify your answers to Problem C.6(b): for each of the four numbers there, print `np.angle` of it and check it against the  $\theta$  you found by hand.

```
thetas = np.pi * np.arange(10) / 10      # 0, pi/10, 2pi/10, ..., 9pi/10
```

**Problem E.7** (Rotation by  $i$ ). (a) For  $z = 3 + 2j$ , print  $i^k z$  for  $k = 0, 1, 2, 3, 4$  and check that the fifth value equals the first, using `np.isclose`. Compare with your hand answers to Problem C.7(a).

(b) Draw 1000 random complex numbers and verify with `np.allclose` that  $|iz| = |z|$  for every one of them.

**Hint.** `zs = rng.normal(size=1000) + 1j*rng.normal(size=1000)` makes all thousand at once, and `np.abs(zs)` takes all thousand moduli at once — no loop needed.

(c) Take the four corners of the unit square, 0, 1,  $1 + i$ ,  $i$ , as an array. Multiply the whole array by  $i$  and print the result. Which corner went where? Multiply by  $i$  three more times and confirm you are back where you started.

(d) Rotate the point (4, 1) by  $90^\circ$  counter-clockwise, by  $180^\circ$ , and by  $\theta = \pi/6$ , each with a single complex multiplication. Print the resulting coordinates as ordinary pairs of real numbers, using `.real` and `.imag`. Check the first two against your hand answers to Problem C.7(d) and (e).

**Problem E.8** (Checking Part C). Print the value of every sub-part of Problem C.2, of Problem C.4(a)–(b), and of Problem C.5(a)–(b), and state in a comment whether each agrees with your hand computation.

Then answer, in a comment at the end of your file, in three or four sentences: which problems on this assignment could NumPy check for you, and which could it not? Name at least one problem from Part B or Part D and say precisely what a computer run would and would not establish about it.

## What NumPy can and cannot do for you

| Part                     | Can NumPy check it?                              |
|--------------------------|--------------------------------------------------|
| A — vectors, computation | Yes, entirely. Do it before you submit.          |
| B — vectors, proof       | No. It can test instances, which is not a proof. |
| C — complex, computation | Yes, entirely. Do it before you submit.          |
| D — complex, proof       | No, for the same reason.                         |
| E — NumPy                | It <i>is</i> the check.                          |

For the drawing sub-parts of Problems A.1, C.1, C.4 and C.7, nothing is being checked by machine — but sketching the picture is often the fastest way to notice that a sign is wrong.

## Optional and ungraded: drawing the pictures

Not required, not graded, and not needed for any later assignment — but if you would like the computer to draw the figures you have been sketching by hand, this is all it takes. It needs one more package:

```
python -m pip install matplotlib
```

**Watch out.** Keep this out of `hw1.py` and `runhw1.py`, in a scratch file of its own. `plt.show()` halts a program until you close the window, so a plot left in your submission stalls the run it is graded from.

Two vectors and their sum, drawn as arrows from the origin:

```
import numpy as np
import matplotlib.pyplot as plt

u = np.array([2.0, 1.0])
v = np.array([1.0, 3.0])

for vec, colour, name in [(u, "C0", "u"), (v, "C2", "v"), (u+v, "C3", "u+v")]:
    plt.arrow(0, 0, vec[0], vec[1], head_width=0.12,
              length_includes_head=True, color=colour)
    plt.text(vec[0], vec[1], f" {name}", color=colour)

plt.axhline(0, color="gray", lw=0.5)
plt.axvline(0, color="gray", lw=0.5)
plt.gca().set_aspect("equal")
plt.grid(alpha=0.3)
plt.show()
```

The four powers of  $i$  times  $z$ , on the complex plane:

```
z = 3 + 2j
points = np.array([z * 1j**k for k in range(4)])

plt.scatter(points.real, points.imag)
for p in points:
    plt.text(p.real, p.imag, f" {p:.0f}")
```

```
plt.gca().set_aspect("equal")
plt.axhline(0, color="gray", lw=0.5)
plt.axvline(0, color="gray", lw=0.5)
plt.grid(alpha=0.3)
plt.show()
```

`points.real` and `points.imag` take the real and imaginary parts of the whole array at once — the same attributes as for a single complex number, applied elementwise.

---

Questions about any problem here belong in the class Discord, where everyone can see the answer. Questions that are specific to your own work — “is my proof of B.4 valid?” — are better brought to office hours, Thursdays 3:00–4:00 PM in Kiely Hall 508. Please email ahead.